INVENTRYX

AI-POWERED WAREHOUSE INTELLIGENCE PLATFORM

AI-Assisted Development & Prompt Engineering Log

YottaFlex.ai · Associate AI Forward Deployed Engineers
Round 2 — 72-Hour Code Challenge

Project	INVENTRYX — AI-Powered Warehouse Intelligence Platform
Challenge	YottaFlex.ai Round 2 Code Challenge
Document type	Development and prompt engineering record
Tagline	Think Smarter. Store Better. Move Faster.

This document summarizes the principal AI-assisted prompts used to support requirements analysis, data engineering, optimization, application integration, testing, branding and final delivery preparation.

1. Document Purpose

This document records the major AI-assisted development activities undertaken during the design, implementation, testing and refinement of INVENTRYX. It captures the principal prompts used to support requirements analysis, application development, business-rule implementation, data generation, testing, optimization, documentation and final delivery preparation.

AI assistance was used as a development aid. Implementation decisions, code integration, execution, testing and final verification were treated as separate engineering activities.

2. Project Overview

INVENTRYX is an AI-powered warehouse intelligence platform designed to optimize SKU placement across warehouse locations using demand velocity, physical dimensions, product compatibility, picking efficiency and replenishment constraints.

- Synthetic warehouse data generation.
- SKU demand and velocity classification.
- Warehouse location and capacity management.
- Intelligent slot candidate ranking.
- Constraint-aware slot assignment.
- Picking-path simulation.
- Replenishment and labor workload analysis.
- Exception detection and management.
- Persistent assignment and event history.
- Operational performance evaluation.
- Interactive web-based monitoring and analysis.

Product tagline: Think Smarter. Store Better. Move Faster.

3. AI-Assisted Development Activities

3.1 — Initial Repository Assessment

> *“Analyze the uploaded project archive against the complete YottaFlex problem statement. Inspect the source code, data models, generated datasets, database schema, business logic, optimization algorithms, validation reports, automated tests and documentation. Identify implemented requirements, missing functionality, potential defects and improvements necessary for a complete submission. Base the assessment on the actual implementation rather than relying exclusively on documentation.”*

3.2 — Requirements Compliance Assessment

> *“Perform a requirement-by-requirement audit of the implementation. Verify the required entities, relationships, primary and foreign keys, deterministic IDs, synthetic data volumes, physical constraints, compatibility rules, demand velocity calculations, candidate slot generation, slot assignments, picking simulation, replenishment workload, exception handling, ground-truth separation, validation and reproducibility. Distinguish fully implemented requirements from partial implementations.”*

3.3 — Data Generation Assessment

> *“Inspect the synthetic data generation pipeline. Verify parent-child dependencies, entity relationships, deterministic identifiers, data distributions, valid foreign-key references, derived calculations and lifecycle consistency. Confirm that the generator supports both starter and full-scale configurations using a fixed random seed.”*

3.4 — Data Quality and Ground-Truth Validation

“Verify the generated datasets against the data dictionary and business rules. Check primary-key uniqueness, foreign-key integrity, allowed values, date ordering, arithmetic reconciliation, duplicate assignments and lifecycle transitions. Ensure that deliberate anomalies are labelled separately through ground-truth records and are not silently introduced into unrelated input data.”

3.5 — Reproducibility Verification

“Execute the data generation process twice using the same configuration and random seed. Compare output row counts, file manifests and SHA-256 checksums. Identify any differences and provide a reproducibility report based on actual execution results.”

3.6 — Business Logic Assessment

“Inspect the warehouse slotting implementation. Verify that SKU placement considers demand velocity, dimensions, weight, storage capacity, hazard compatibility, temperature requirements, picking frequency and replenishment workload. Ensure that infeasible locations are rejected and that valid candidate locations are ranked using documented business rules.”

3.7 — Optimization Evaluation

“Evaluate the slotting optimization methodology. Compare the baseline and optimized allocation strategies using picking distance, picker throughput, golden-zone utilization and replenishment labor. Verify that both strategies are evaluated using comparable demand and workload conditions. Document the optimization methodology, assumptions and limitations.”

3.8 — Exception Scenario Testing

“Validate the required negative scenarios: SKU capacity exceeded, incompatible product co-location, duplicate active location assignments, blocked-aisle pick paths and replenishment labor-capacity violations. Verify that each scenario produces an appropriate issue classification, evidence and ground-truth label.”

3.9 — End-to-End Application Assessment

“Inspect the complete application architecture, including the Python backend, web interface, business logic, data processing pipeline and API endpoints. Verify that the application supports warehouse monitoring, SKU inspection, slot proposals, replenishment analysis, exception management and operational reporting. Identify integration issues without unnecessarily restructuring the existing implementation.”

3.10 — Application Execution Guide

“Provide complete Windows-based Visual Studio Code execution instructions. Include Python environment setup, dependency installation, starter and full dataset generation, validation, automated testing, application startup and browser access. Supply exact commands in the correct execution order.”

3.11 — Proposal Persistence Assessment

“Inspect interactive slot proposal handling. Verify durable persistence, append-only event history, proposal recovery after server restarts, approval and withdrawal workflows, planner identification and assignment conflict checks. Ensure that new planner decisions do not overwrite historical records.”

3.12 — Effective-Date Validation

“Inspect effective-date conflict handling for warehouse slot assignments. Verify overlapping and non-overlapping assignment periods, future-dated proposals, open-ended assignments and boundary conditions. Correct any identified defects and add regression tests without changing unrelated business logic.”

3.13 — INVENTRYX Rebranding

“Rename the existing application to INVENTRYX across the entire codebase. Update frontend branding, backend metadata, navigation, page titles, documentation, reports, API descriptions, startup messages and project metadata. Use the product description 'AI-Powered Warehouse Intelligence Platform' and the tagline 'Think Smarter. Store Better. Move Faster.' Preserve all existing application functionality, business rules, datasets, APIs and optimization algorithms.”

Branding specifications

Element	Specification
Application name	INVENTRYX
Product category	Warehouse Intelligence Platform
Tagline	Think Smarter. Store Better. Move Faster.
Primary colour	Deep Navy Blue
Secondary colour	Electric Blue
Accent colours	Cyan and Teal
Design direction	Professional Enterprise SaaS

3.14 — Automated Test Execution

“Execute the complete automated test suite against the latest project build. Review test failures, skipped tests and execution errors. Verify the backend, application APIs, data generation, business rules, persistence, effective-date handling and frontend integration. Report actual test results and do not claim successful execution without evidence.”

3.15 — Full Dataset Validation

“Execute the full dataset validation process. Verify entity volumes, primary and foreign keys, allowed values, compatibility constraints, assignment integrity, event history, ground-truth consistency and derived calculations. Report the exact number of successful and failed validation checks.”

3.16 — Web Application Verification

“Start the application using the provided starter dataset. Verify the homepage, static frontend resources and major API endpoints. Check warehouse summaries, SKU information, slot proposals, data quality, exceptions and replenishment analytics. Identify runtime errors and incomplete functionality.”

3.17 — Repository Cleanup

“Inspect the complete repository and identify unnecessary development artifacts, temporary files, virtual environments, Python caches, duplicate archives and bulky generated datasets. Preserve all essential source code, tests, starter data, ground-truth records, SQL exports, documentation and reproducibility reports. Ensure that cleanup does not affect application execution.”

3.18 — GitHub Submission Audit

“Review the final repository structure and .gitignore configuration. Verify that unnecessary files, generated full-scale datasets, temporary outputs, credentials and duplicate archives are excluded. Confirm that all required source files, documentation, tests and configuration files are retained. Provide the exact Git commands for committing and pushing the final project to GitHub.”

3.19 — Final Acceptance Assessment

“Perform a final assessment of the project against the original YottaFlex challenge requirements. Review the source code, data generation, optimization, exception handling, persistence, validation, reproducibility, evaluation metrics, web application and documentation. Clearly identify any remaining limitations and provide a final submission-readiness assessment based on executed verification results.”

4. Verification and Engineering Principles

- **Requirements traceability:** Implementation decisions were evaluated against the original business requirements.
- **Data integrity:** Relational dependencies and deterministic identifiers were preserved.
- **Business-rule consistency:** Derived outcomes and exception labels were generated from documented rules.
- **Historical traceability:** State changes were maintained through explicit event records.
- **Reproducibility:** Fixed-seed generation and checksum comparisons were used to assess consistency.
- **Test-driven verification:** Automated tests and validation commands were executed against the implementation.
- **Controlled modifications:** Existing architecture and business logic were preserved when introducing improvements.
- **Transparent reporting:** Test results, limitations and incomplete verification activities were distinguished clearly.

5. Final Project Deliverable

INVENTRYX — AI-Powered Warehouse Intelligence Platform

The final application brings together synthetic warehouse data engineering, demand-aware SKU slotting, physical and compatibility constraints, picking and replenishment simulation, exception management, persistent operational records and measurable optimization outcomes.

The development process combined requirements analysis, iterative implementation, automated testing, data validation and application-level verification to produce a warehouse slotting solution for the YottaFlex.ai Round 2 Code Challenge.

Document: AI-Assisted Development & Prompt Engineering Log
Project: INVENTRYX
Challenge: YottaFlex.ai — Associate AI Forward Deployed Engineers
Document status: Final Submission Documentation